第 2 天 · 1 小时专属学习计划：Python 核心数据类型详解

第 2 天 · 1 小时专属学习计划（0 基础 · VS Code 适配 · 贴合爬虫 / 网安方向）

✅ **今日主题**：Python 核心数据类型 + 类型转换 + 字符串高频操作（爬虫 / 网络自动化 / 网安脚本 90% 都会用到的核心基础）
⏱️ **总时长**：严格 60 分钟，分时段精准控制
💡 **学习目标**：彻底解决新手最容易踩的「类型报错」坑，掌握字符串的核心用法，能写出带用户输入、自动计算的智能小程序

00:00-00:05 昨日快速回顾（5 分钟）

先快速衔接第 1 天的核心内容，确保你不会断档：

1. `print()`：程序的输出功能，`\n` 实现换行
2. `变量名 = 数据`：用变量存储内容
3. `input()`：获取用户的键盘输入
4. `#`：单行注释，程序不会执行
5. 你踩过的坑：变量和字符串不能直接连写，字符串和数字不能直接拼接 / 计算

今天我们就解决第 1 天的遗留问题：**不用写死出生年份，让用户自己输入，程序自动计算年龄**，这也是所有脚本逻辑的基础。

00:05-00:35 核心知识点逐行讲解（30 分钟）

每一个知识点都配套代码示例 + 逐行解释 + 后续使用场景，0 基础完全能懂，学完马上能用。

模块 1：3 种核心数据类型（新手必懂）

Python 里所有数据都有「类型」，不同类型能干的事不一样，你后续写脚本 99% 只会用到这 3 种，也是第 1 天已经接触过的：

类型	全称	说明	示例

`str`	字符串	用双引号 `""` / 单引号 `''` 包裹的内容，`input()` 获取的内容永远是字符串	`name = "张三"`、`url = "https://github.com"`
`int`	整数	不带小数点的数字，用于数学计算	`birth_year = 1994`、`port = 80`
`float`	浮点数	带小数点的数字，用于精确计算	`height = 1.78`、`delay = 0.5`

必学工具：`type()` 查看变量类型

90% 的新手报错，都是因为「数据类型不对」，用 `type()` 可以直接查看变量的类型，排查报错超级好用。
在 VS Code 的 `day2.py` 里输入这段代码，按 `Ctrl+F5` 运行：

代码块

```
# 用type()查看变量的类型
name = "荔枝Python"
birth_year = 1994
height = 1.78

print(type(name))        # 输出 <class 'str'> 字符串类型
print(type(birth_year)) # 输出 <class 'int'> 整数类型
print(type(height))      # 输出 <class 'float'> 浮点数类型
```

新手必踩的天坑（提前避坑）

`input()` 获取的所有内容，哪怕你输入的是数字，类型永远是字符串 `str`！
直接运行下面这段代码，你会看到报错：

代码块

```
# 新手必踩坑示例（运行会报错）
birth_year = input("请输入你的出生年份：")
print(type(birth_year)) # 你输入1994，这里还是输出<class 'str'>，不是数字！
age = 2026 - birth_year # 报错！数字不能直接减字符串
```

报错原因：你输入的 `1994`，在程序里是带引号的字符串 `"1994"`，不是能计算的数字，这就是我们接下来要解决的问题。

模块 2：类型转换（核心必学）

把一种数据类型转换成另一种，解决上面的报错问题，也是写脚本时每天都会用到的功能，核心 3 个函数：

1. `int(内容)`：把符合数字格式的内容转换成整数
2. `str(内容)`：把任意内容转换成字符串
3. `float(内容)`：把符合数字格式的内容转换成浮点数

1. 解决上面的报错：input 输入数字计算

代码块

```
# 正确写法1：分步转换
birth_year = input("请输入你的出生年份：")
birth_year = int(birth_year) # 把字符串"1994"转成数字1994
age = 2026 - birth_year
print("你的年龄是：", age)

# 正确写法2：一行搞定（推荐日常用）
birth_year = int(input("请输入你的出生年份："))
age = 2026 - birth_year
print("你的年龄是：", age)
```

2. 字符串和数字拼接：用 str () 转换

代码块

```
age = 32
# 错误写法：print("我的年龄是"+age) 字符串不能直接+数字
# 正确写法：用str()把数字转成字符串
print("我的年龄是："+str(age))
```

后续使用场景

- 爬虫：拼接分页网址 `"https://xxx.com/page/"+str(page_num)`
- 网安：拼接 IP 和端口 `ip+":"+str(port)`
- 自动化：把命令执行的数字结果转成文本保存

模块 3：字符串高频操作（爬虫 / 网安必备）

字符串是你后续写脚本用得最多的类型：处理网址、爬取的文本、设备命令输出、请求头内容，全靠字符串操作。今天只学最高频、必用的 5 个，学完马上能落地。

1. 字符串拼接：用 `+` 号

把多个字符串连在一起，拼接网址、输出文本必用

代码块

```
# 字符串拼接
first_name = "Yan"
last_name = "618"
full_name = last_name + first_name
print(full_name) # 输出618Yan

# 带提示文字拼接
print("你的完整昵称是：" + full_name)
```

⚠ 注意：`+` 号拼接时，所有内容必须都是字符串，数字要先用 `str()` 转换。

2. 字符串切片：提取指定内容

从长字符串里提取你想要的部分，爬虫提取网址里的用户名、网安提取 IP 里的网段，超级常用。
语法： `字符串[起始索引:结束索引]`

- Python 索引从 `0` 开始，不是 1
- 左闭右开：包含起始位置的内容，不包含结束位置的内容

代码块

```
# 切片示例（用你的GitHub地址举例）
url = "https://github.com/daloyanf/python-.git"

# 提取网址里的用户名
user_name = url[19:27]
print(user_name) # 输出daloyanf

# 常用简写
print(url[:8]) # 从开头取到第8位，输出https://
print(url[19:]) # 从第19位取到结尾，输出daloyanf/python-.git
```

3. `strip()`：去掉首尾多余空格 / 换行

用户输入时经常会不小心打空格，爬虫爬下来的内容也会有多余空白，这个函数能一键清理，必用！

代码块

```python
# strip() 去掉首尾空格
name = input("请输入你的名字：") # 用户不小心输入了  张三  （前后有空格）
name = name.strip() # 一键去掉前后空格
print("你好，"+name)

# 进阶：input的时候直接处理
name = input("请输入你的名字：").strip()
```

4. `lower()` / `upper()`：大小写转换

处理网址、请求头、用户输入时，避免大小写不一致导致的问题，比如 `HTTPS` 和 `https` 在程序里是两个不同的内容。

代码块

```python
# 大小写转换
url = "HTTPS://GITHUB.COM"
url = url.lower() # 转成全小写
print(url) # 输出https://github.com

name = "yan"
name = name.upper() # 转成全大写
print(name) # 输出YAN
```

5. `replace()`：替换指定内容

爬虫里替换掉文本里的无效字符、自动化里修改命令参数，超级常用。
语法： `字符串.replace(要替换的内容， 替换成的内容)`

代码块

```python
# replace() 替换内容
text = "我的出生年份是1994"
new_text = text.replace("1994", "1995")
print(new_text) # 输出我的出生年份是1995

# 爬虫场景：去掉文本里的换行符
content = "这是爬取的\n文本内容"
content = content.replace("\n", "")
print(content) # 输出这是爬取的文本内容
```

00:35-00:55 完整实操练习（20 分钟）

要求

1. 在 VS Code 里新建文件，保存为 `day2.py`
2. **亲手手打**下面的完整代码（不要复制粘贴，手打才能记住）
3. 按 `Ctrl+S` 保存， `Ctrl+F5` 运行，逐行理解每一句的作用
4. 用到今天学的所有知识点，衔接你第 1 天的代码，升级成智能版本

代码块

```python
# ========== 第2天 完整练习代码 ==========
# 1. 获取用户输入，同时用strip()去掉首尾多余空格，避免无效输入
name = input("请输入你的名字：").strip()
# 把输入的出生年份直接转成整数，用于后续计算
birth_year = int(input("请输入你的出生年份：").strip())

# 2. 自动计算年龄
current_year = 2026
age = current_year - birth_year

# 3. 两种格式化输出方式（对比学习）
# 方式1：用+号拼接，数字必须用str()转成字符串
print("===== 方式1 拼接输出 =====")
print("你好，" + name + "！你今年" + str(age) + "岁了。")

# 方式2：f-string（推荐日常用，不用转类型，更简洁）
print("\n===== 方式2 f-string输出 =====")
print(f"你好，{name}！你今年{age}岁了。")

# 4. 字符串高频操作演示
print("\n===== 字符串操作 =====")
# 把名字转成全大写
name_upper = name.upper()
print(f"你的名字大写格式：{name_upper}")
# 切片提取名字的第一个字
first_char = name[0:1]
print(f"你名字的第一个字：{first_char}")

# 5. 查看变量类型，验证今天学的知识点
print("\n===== 变量类型查看 =====")
print(f"name变量的类型：{type(name)}")
print(f"birth_year变量的类型：{type(birth_year)}")
print(f"age变量的类型：{type(age)}")
```

逐行核心解释

1. `input().strip()`：获取输入的同时直接清理首尾空格，避免用户误输入导致的问题
2. `int(input())`：直接把用户输入的年份转成整数，解决了不能计算的报错问题
3. 两种输出方式对比：`f-string` 不用手动转类型，写法更简洁，后续写代码优先用这个
4. 字符串操作和类型查看，帮你巩固今天学的所有核心知识点

00:55-01:00 今日总结 + 避坑指南（5 分钟）

✅ 今日必须记住的核心

1. 3 种核心数据类型：`str` 字符串、`int` 整数、`float` 浮点数
2. `input()` 获取的所有内容，永远是 `str` 类型，要计算必须用 `int()` / `float()` 转换
3. 3 个类型转换函数：`int()` 转整数、`str()` 转字符串、`float()` 转浮点数
4. 字符串高频操作：`+` 拼接、`[]` 切片、`strip()` 去空格、`lower()` / `upper()` 转大小写、`replace()` 替换
5. 用 `type(变量)` 可以查看数据类型，90% 的新手报错都能靠它排查

⚠ 0 基础必避坑

1. 字符串和数字不能直接做加减运算，也不能直接用 `+` 拼接，会报错
2. 字符串索引从 `0` 开始，不是 1，切片是左闭右开，不要超出字符串长度
3. `int()` 只能转数字格式的字符串，`int("1994")` 可以，`int("张三")` 会直接报错
4. 每次改完代码，必须按 `Ctrl+S` 保存后再运行

今日 Git 提交命令（直接复制运行）

代码块

```
# 1. 把今天的代码加入Git版本管理
git add .
# 2. 提交到本地Git，写清提交说明
git commit -m "第2天学习完成：数据类型+类型转换+字符串高频操作"
# 3. 推送到GitHub（如果还是网络报错，不用管，本地提交成功就不影响学习）
git push
```

运行成功、提交完成后，回复我 \|「第 2 天全部完成」\|，我会给你安排第 3 天的学习内容（条件判断 `if/else`，写出第一个带逻辑判断的小程序，也是所有自动化 / 攻防脚本的核心逻辑）。

（注：文档部分内容可能由 AI 生成）